

ZeroClick

A Predictive Household Restock Forecaster

Prepared By:

Arjun Rathore

2024A4PS0691H

Institution:

Birla Institute of Technology and Science, Hyderabad Campus
Hyderabad, Telangana, India

Date: July 21, 2026

Contents

1	Introduction	2
2	Phase 1: Multi-Tenant Data Structures and Memory Management	2
2.1	Data Generation Engine	2
2.2	The $O(1)$ Master Hash Map	4
3	Phase 2: Dynamic AI Forecasting Engine	4
3.1	Weighted Moving Average (WMA) Formulation	4
3.2	The Isolated Max-Heap Data Structure ($O(\log n)$)	4
4	Phase 3: Frontend Architecture & UI Abstractions	5
4.1	Graceful Degradation and Abstractions	5
4.2	The Swipeable Action Container	6
5	Phase 4: System Resilience & Edge Cases	6
5.1	Preventing Race Conditions	6
5.2	Optimistic UI Updates	7
6	Errors Faced and Algorithmic Resolutions	7
6.1	The "Infinite Restock" Loop	7
6.2	Memory Profiling & Session Isolation	8
7	How to Run the Project	8
8	Conclusion	8

1 Introduction

The most repetitive and cognitively draining e-commerce task is the procurement of everyday consumables such as groceries, toiletries, and household supplies. Consumers consistently experience "restock fatigue," frequently forgetting to replenish essential items until they are entirely depleted. Standard e-commerce paradigms attempt to solve this via subscription models (e.g., "Subscribe & Save"), but these models are rigid, leading to either a surplus of unwanted items or premature shortages due to changing consumer habits.

The "Zero-Click" Household Restock Forecaster was developed to revolutionize this flow. Instead of waiting for manual user intervention, this project implements a FAANG-grade predictive engine that calculates exactly when a user will run out of a product based on historical purchase velocity. By marrying an algorithmic C++ backend with a highly resilient React frontend, the system autonomously queues necessary items for dispatch, requiring zero clicks from the user unless they wish to intervene.

This report details the architectural decisions, algorithmic complexities, and system resilience testing involved in engineering this Full-Stack application.

2 Phase 1: Multi-Tenant Data Structures and Memory Management

The primary objective of Phase 1 was to construct a robust data foundation capable of simulating the massive scale demanded by top-tier tech organizations (e.g., Flipkart, Amazon) and to establish a backend architecture that prevents cross-user data bleeding under high load.

2.1 Data Generation Engine

To accurately stress-test the algorithms, a massive synthetic dataset was required. Rather than utilizing sluggish Python scripts, a localized C++ data generator was developed leveraging the `nlohmann/json` library. This engine initializes 10,000 distinct mock users, assigning varied personas (e.g., "Single Professional," "Large Family") and simulating a 6-month (180-day) purchase history for each.

```
1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <string>
5 #include <random>
6 #include "json.hpp"
7
8 using namespace std;
9 using json = nlohmann::json;
10
11 class DataGen {
12 private:
```

```

13     int numUsers;
14     vector<string> types;
15     vector<string> items;
16     random_device rd;
17     mt19937 gen;
18
19     int getRand(int min, int max) {
20         uniform_int_distribution<> dist(min, max);
21         return dist(gen);
22     }
23
24 public:
25     DataGen(int n) {
26         numUsers = n;
27         gen = mt19937(rd());
28         types = {"Single", "Family"};
29         items = {"Coffee", "Paste", "Milk", "Soap"};
30     }
31
32     void build(string file) {
33         json db;
34         cout << "Making data for " << numUsers << " users...\n";
35
36         for (int i = 1; i <= numUsers; i++) {
37             string uid = "U" + to_string(i);
38             string type = types[getRand(0, types.size() - 1)];
39             json history = json::array();
40             int buys = getRand(15, 60);
41
42             for (int j = 0; j < buys; j++) {
43                 json item;
44                 item["name"] = items[getRand(0, items.size() - 1)];
45                 item["day"] = getRand(1, 180);
46                 history.push_back(item);
47             }
48
49             json user;
50             user["type"] = type;
51             user["history"] = history;
52             db[uid] = user;
53         }
54
55         ofstream out(file);
56         if (out.is_open()) {
57             out << db.dump(4);
58             out.close();
59         }
60     }
61 };
62
63 int main() {

```

```

64     DataGen gen(10000);
65     gen.build("data.json");
66     return 0;
67 }

```

Listing 1: Mock Data Generator (data_gen.cpp)

2.2 The $O(1)$ Master Hash Map

A naive backend reads from a disk-based database for every API request, introducing massive I/O latency. To achieve near-instantaneous response times, the entire 10,000-user JSON payload is loaded directly into the C++ server’s RAM upon initialization.

The data is mapped into a `std::unordered_map` (Hash Map). The key represents the `user_id` (e.g., "U7"), and the value is the parsed JSON history. When a frontend request arrives, the backend resolves the user’s isolated data environment in strictly $O(1)$ constant time, bypassing all database bottlenecks.

3 Phase 2: Dynamic AI Forecasting Engine

The core intelligence of the application resides in Phase 2. The objective was to replace static subscriptions with a dynamic Time-Series Algorithm implemented entirely from scratch in C++.

3.1 Weighted Moving Average (WMA) Formulation

To determine a user’s restock velocity, the system cannot rely on a simple mean average, as human habits evolve (e.g., a user might start drinking coffee more frequently in winter). The backend implements a Weighted Moving Average (WMA), assigning higher mathematical significance to recent purchases.

The algorithm calculates the interval (in days) between historical purchases. It then iterates through these intervals, multiplying recent gaps by ascending weights. The resulting projected runout date dynamically adjusts to the user’s latest behavioral shifts.

3.2 The Isolated Max-Heap Data Structure ($O(\log n)$)

Once the WMA predicts a runout day, the system calculates an `urgencyScore`. To determine which items require immediate UI rendering, the system must sort the catalog.

Using a standard array sort would incur an $O(n \log n)$ time complexity penalty. Instead, the backend pushes each calculated score into a `std::priority_queue` configured as a Max-Heap. Insertion operates in $O(\log n)$ time, and the Heap inherently bubbles the most critical items to the top. The top items are then extracted in $O(1)$ time.

Furthermore, a specific algorithmic override was introduced: if a user purchased an item within the last two days, the urgency is hard-coded to -999 , immediately purging it from the active queue.

```

1 // WMA Formula Calculation
2 for (int i = 0; i < n; i++) {
3     int weight = i + 1;
4     wma += intervals[i] * weight;
5     weightSum += weight;
6 }
7 wma /= weightSum;
8
9 int lastDay = days.back();
10 int projectedRunoutDay = lastDay + (int)wma;
11 int urgencyScore = 180 - projectedRunoutDay;
12
13 // ALGORITHM FIX: If bought in the last 2 days, hide from UI
14 if (180 - lastDay <= 2) {
15     urgencyScore = -999;
16 }
17
18 // Push to Max-Heap O(log n)
19 if (urgencyScore > -3) {
20     maxHeap.push({urgencyScore, itemName});
21 }
22
23 // Extract top 3 most urgent items in O(1)
24 json forecast = json::array();
25 int count = 0;
26 while (!maxHeap.empty() && count < 3) {
27     forecast.push_back(maxHeap.top().second);
28     maxHeap.pop();
29     count++;
30 }

```

Listing 2: WMA and Max-Heap Logic (api_server.cpp)

4 Phase 3: Frontend Architecture & UI Abstractions

The UI was engineered using React and Tailwind CSS v4, adhering strictly to a "gritty, minimalist" design language. The interface is optimized to act as a distraction-free action center.

4.1 Graceful Degradation and Abstractions

High-concurrency systems inevitably experience latency. To prevent layout shifts (Cumulative Layout Shift) and UI freezing, reusable `<SkeletonCard />` abstractions were developed. These components utilize CSS animations to pulse while awaiting the C++ backend response, maintaining the exact DOM dimensions of the final loaded data.

4.2 The Swipeable Action Container

Because the system employs "Zero-Click" automatic ordering, users require a psychological safety mechanism to override AI decisions. A custom, gesture-based mobile interaction was engineered entirely from scratch, avoiding heavy third-party animation libraries.

Utilizing React `useRef` and touch event listeners, the `<ProductCard />` component mathematically tracks pixel deltas. If a user swipes the card left past a threshold of -110 pixels, the restock is aborted.

```
1 const ProductCard = ({ itemName, onCancelRestock, onInstantCheckout }) => {
2   const [offset, setOffset] = useState(0);
3   const [isSwiped, setIsSwiped] = useState(false);
4   const startX = useRef(0);
5   const currentX = useRef(0);
6
7   const handleTouchStart = (e) => {
8     startX.current = e.touches ? e.touches[0].clientX : e.clientX;
9
10    const handleTouchMove = (e) => {
11      if (isSwiped) return;
12      currentX.current = e.touches ? e.touches[0].clientX : e.clientX;
13      const diff = currentX.current - startX.current;
14      if (diff < 0 && diff > -160) setOffset(diff); // Only allow left swipe
15    };
16
17    const handleTouchEnd = () => {
18      if (offset < -110) {
19        setIsSwiped(true);
20        setOffset(-1000);
21        onCancelRestock(itemName);
22      } else {
23        setOffset(0); // Snap back if threshold not met
24      }
25    };
26    // ... JSX returning dynamic transform translate styles ...
27  }
```

Listing 3: Custom Swipe Gesture Logic (App.jsx)

5 Phase 4: System Resilience & Edge Cases

The final phase of engineering tested the application against rapid context switching and UI stress testing to prove FAANG-level resilience.

5.1 Preventing Race Conditions

When a user rapidly toggles between Active Profiles, asynchronous fetch requests overlap. If User A's API request is delayed by the network, and User B's request returns instantly, User

A's stale data could eventually overwrite the UI, resulting in a severe data-bleed race condition.

To counter this, a strict boolean `ignore` flag was implemented inside the React `useEffect` cleanup function. If the component unmounts or the dependency array updates before a fetch concludes, the late-arriving data is intentionally discarded.

5.2 Optimistic UI Updates

To simulate an instantaneous production environment, network latency during order execution is visually bypassed. When a user executes an order, the React state immediately purges the item from the localized queue array, vanishing the UI card instantly. The actual HTTP POST request to the C++ server happens asynchronously in the background.

```
1 const processPurchase = async (itemsToBuy) => {
2   const itemNamesToBuy = itemsToBuy.map(i => (i.name || i).toLowerCase());
3
4   // OPTIMISTIC UI: Instantly clear items from screen before server responds
5   setForecastData(prev => ({
6     ...prev,
7     forecast: prev.forecast.filter(item => !itemNamesToBuy.includes(item.
8       toLowerCase()))
9   }));
10
11   try {
12     for (const item of itemsToBuy) {
13       await fetch('http://localhost:8080/api/buy', {
14         method: 'POST',
15         headers: { 'Content-Type': 'application/json' },
16         body: JSON.stringify({ uid: activeUserId, item: item.name || item,
17           day: 180 })
18       });
19     }
20   } catch (err) {
21     setToast({ message: 'Network Error: Backend Sync Failed.', type: 'cancel'
22       });
23   }
24 }
```

Listing 4: Optimistic State Purge (App.jsx)

6 Errors Faced and Algorithmic Resolutions

Several critical logical bugs emerged during full-stack integration.

6.1 The "Infinite Restock" Loop

Issue: After placing an order, the UI successfully synced with the C++ backend, adding a new purchase record for Day 180. However, upon refreshing the API, the backend *still* recommended the newly purchased item.

Resolution: The Max-Heap was blindly returning the top two items regardless of their actual urgency score. Even with an urgency score of -15 (indicating ample supply), if it was the highest relative value, it was returned. The C++ logic was amended with an explicit threshold guard: `if (urgencyScore > -3) { maxHeap.push(...); }`, ensuring only genuinely depleted items populate the queue.

6.2 Memory Profiling & Session Isolation

Issue: Writing automated orders directly to `data.json` rapidly corrupted the static mock database, rendering future QA testing impossible.

Resolution: The C++ backend was refactored into a "Session-Only" memory model. The `POST /api/buy` endpoint updates the `std::unordered_map` residing purely in active RAM. This provides the React UI with real-time recalculations while preserving the integrity of the underlying `.json` payload across server reboots.

7 How to Run the Project

The architecture is decoupled into two independent execution environments.

1. Backend Execution (C++):

1. Ensure a standard `g++` compiler is installed (MinGW for Windows).
2. Generate the mock payload: `g++ data_gen.cpp -o datagen && ./datagen`
3. Compile the API: `g++ api_server.cpp -o server -lws2_32` (Windows) or `-pthread` (Unix).
4. Execute the server binary on Port 8080.

2. Frontend Execution (React/Vite):

1. Navigate to the `zero-click-ui` directory containing `App.jsx` and the Vite configurations.
2. Install dependencies: `npm install`
3. Launch the dev server: `npm run dev` (Initializes on Port 5173).

8 Conclusion

The Zero-Click Household Restock Forecaster proves that heavy, multi-tenant predictive logic can be efficiently isolated and executed at scale. By leveraging C++ for mathematically rigorous, $O(1)$ Hash Map routing and $O(\log n)$ Max-Heap sorting, the system easily offloads computational burden from the client. Combined with a React frontend that masterfully handles race conditions and optimistic UI purging, the project achieves an enterprise-grade standard of performance and reliability.